

CAPSTONE PROJECT · PRODUCT REQUIREMENTS DOCUMENT

Mira: AI-Powered Project Intelligence Assistant

Product Requirements Document (PRD)

This PRD specifies what Mira does: its architecture (a Router over four specialist agents), the functional requirements and acceptance criteria for each capability, the system-prompt structure and grounding guardrails, the delivery integrations, and the 12 baseline test cases that define correct behavior.

Prepared by: Praveen Kumar Jatta — Technical Program Management

Program: Applied Agentic AI for PMs/TPMs · Capstone: Mira

Platform: n8n (mandatory) · LangFlow (secondary) · gpt-4o-mini · Langfuse

Date: Friday, August 7th, 2026

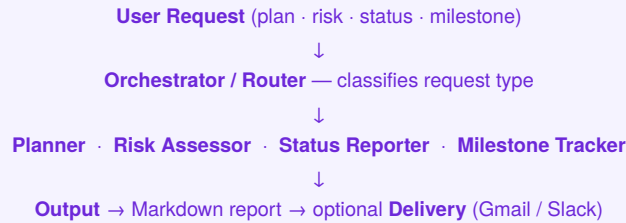
Document: 2 of 4 — BRD · PRD · Iteration Plan · Ground-Truth Dataset

Contents

Section	Page
1. Product Overview & Architecture	3
2. System Prompt Structure & Grounding Guardrail	3
3. Functional Requirements — the Agents	4
4. Delivery & Integration Requirements	6
5. Data / Grounding Sources	6
6. Baseline Test Cases (T1–T12) & Acceptance Criteria	7
7. Non-Functional Requirements	8

1. Product Overview & Architecture

Mira is a multi-agent assistant built on the **Router / Dispatcher** pattern. A user request arrives in natural language; an Orchestrator classifies its intent and dispatches to one of four specialist agents. Each agent does one job, is grounded in the ABCDE Ltd. project data, and carries a guardrail that forces a request for clarification when input is insufficient.



Why Router: Mira's requests are genuinely different types ("generate a plan" vs "what are the risks" vs "give me a Sprint 3 status" vs "which milestones are at risk"). A router keeps each agent focused and independently testable — the right pattern for varied request types, as opposed to a sequential pipeline (which would force every request through all agents) or hierarchical (heavier than needed).

2. System Prompt Structure & Grounding Guardrail

Every agent's system prompt follows the five-part structure taught in the course, and the fifth part — the vague-input rule — is the single most important line for trustworthiness.

Part	Content
01 Role	One sentence: what this agent is (e.g., "You are a project planning specialist.")
02 Input	Exactly what it receives (the request + the relevant CSV/text data)
03 Output	Exact format and fields expected (e.g., phases with milestones and deliverables)
04 Rules	What it must never invent (no milestones/tasks/risks not in the data)
05 Vague rule	The guardrail: "If the input is too vague to determine X, say so and ask for detail. Do NOT invent X. Use only information explicitly present in the provided data."

The guardrail (applied verbatim in every agent): "If the input is too vague or lacks the data needed, respond that more information is required and state what is missing. Do NOT fabricate plans, risks, statuses, milestones, counts, names, or dates. Ground every statement in the provided project files."

3. Functional Requirements — the Agents

3.0 Orchestrator / Router

ID	Requirement
FR-R1	Classify each request into one of: plan · risk · status · milestone
FR-R2	Dispatch to the matching specialist agent
FR-R3	If intent is ambiguous, ask the user to clarify rather than guessing

3.1 Planner Agent — Project Plan Generation

ID	Requirement
FR-P1	Generate a phased project plan grounded in the project description + timeline CSV
FR-P2	Each phase includes: phase name, key activities, milestone/deliverable, timeline — traceable to source
FR-P3	Must NOT invent phases, milestones, or goals not present in the source data
FR-P4	If the request lacks scope (e.g., "build a chatbot," "2-week project no details"), respond that detail is needed — do not generate a fabricated plan

Acceptance: T1 → grounded 8-phase plan mirroring the timeline. T2 & T9 (vague) → request for more information, no invented plan.

3.2 Risk Assessor Agent — Risk Assessment

ID	Requirement
FR-K1	Produce a categorized risk matrix from the risk CSV + description
FR-K2	Each risk: category, description, impact, mitigation — drawn from the data, project-specific
FR-K3	Must NOT emit generic IT-project boilerplate risks absent from the data
FR-K4	Support "top N risks" queries (e.g., top 3) using the provided risk set
FR-K5	If input is vague ("new project starting soon"), request detail — do not invent risks

Acceptance: T3 → categorized matrix from R01–R10. T7 → top 3 risks selected from the data. T4 (vague) → request for more information.

3.3 Status Reporter Agent — Weekly Status Report

ID	Requirement
FR-S1	Read the task board CSV and report tasks grouped by status with correct counts
FR-S2	Reference real task names and surface blockers explicitly (e.g., T024 — Security review, BLOCKED)
FR-S3	Support sprint-scoped reports (e.g., Sprint 3) and overall status summaries
FR-S4	Must NOT fabricate task statuses, counts, or progress percentages
FR-S5	If no task data is supplied ("things are going fine"), request the data — do not invent a status

Acceptance: T5 → Sprint 3 status with correct tasks. T10 → correct counts (5 Done, 3 In Progress, 1 Blocked, 16 To Do). T6 (no data) → request for data.

3.4 Milestone Tracker Agent — Milestone / Blocked-Task Detection (extended)

ID	Requirement
FR-M1	Compare timeline vs task-board progress to flag at-risk / blocked items
FR-M2	Correctly surface blocked tasks — T024 (Security review, BLOCKED) — with reason
FR-M3	Identify upcoming milestones within a requested window (e.g., next 2 weeks) from the timeline
FR-M4	Must NOT invent milestones not present in the timeline

Acceptance: T8 → identifies T024 as blocked. T11 → upcoming milestones from the real timeline.

4. Delivery & Integration Requirements

Per the business requirement, Mira must **deliver** outputs, not just generate them.

ID	Requirement
FR-D1	Generate a stakeholder update email summarizing sprint progress (T12), grounded in the task board
FR-D2	Deliver the stakeholder update via Gmail (native n8n node)
FR-D3	Deliver / post the report or update to Slack (native n8n node)
FR-D4	Delivery is triggered after the specialist agent produces its grounded output

Acceptance: T12 → a professional stakeholder update referencing real Sprint 2 tasks, deliverable to Gmail and Slack.

5. Data / Grounding Sources

File	Contents	Used by
<code>project_description.txt</code>	ABCDE Ltd. AI adoption overview, goals, phases	Planner, Risk Assessor
<code>project_timeline.csv</code>	8 phases × 24 weeks with milestones/deliverables	Planner, Milestone Tracker
<code>project_risks.csv</code>	10 risks (R01–R10): category, impact, mitigation	Risk Assessor
<code>sample_task_board.csv</code>	25 tasks (T001–T025): status, assignee, sprint, due date; T024 = Blocked	Status Reporter, Milestone Tracker

6. Baseline Test Cases (T1–T12) & Acceptance Criteria

#	Input	Type	Expected behavior (acceptance)
T1	Project plan for ABCDE Ltd.	Real	Grounded phased plan from timeline
T2	Plan for "build a chatbot"	Vague	Request more info; no fabricated plan
T3	Risk assessment for ABCDE Ltd.	Real	Categorized matrix from risk data
T4	Risk for "new project starting soon"	Vague	Request more info; no invented risks
T5	Weekly status, Sprint 3	Real	Sprint 3 tasks, correct statuses
T6	Status for "things are going fine"	Vague	Request task data; no fabricated counts
T7	Top 3 risks	Real	3 risks selected from the risk data
T8	Blocked / at-risk tasks	Real	Identifies T024 (Security review, Blocked)
T9	Plan for 2-week project, no details	Vague	Request more info; no fabricated plan
T10	How many done / in progress / to do	Real	Correct counts: 5 / 3 / 16 (+1 Blocked)
T11	Milestones in next 2 weeks	Real	Real upcoming milestones from timeline
T12	Stakeholder update email, Sprint 2	Real	Grounded update; delivered via Gmail/Slack

The four vague cases (T2, T4, T6, T9) are the trust test. Passing them — by asking for information rather than fabricating — is the clearest evidence Mira is grounded and safe. These carry the most weight in the acceptance review.

7. Non-Functional Requirements

Category	Requirement
Groundedness	Every factual claim traces to the provided data; zero fabricated facts
Consistency	Stable output across repeated runs (verified across 10 runs in evaluation)
Observability	Per-agent input/output, tokens, latency, and cost visible in Langfuse / n8n logs
Cost	~3,500 tokens/run on gpt-4o-mini $\approx$ \\$.0005/run; ~\\$0.20/user/month at ~15 runs/day
Latency	Report generated in seconds; acceptable for interactive PM use
Maintainability	One job per agent; prompts editable independently; workflow exported as JSON

Build order (per the playbook): get one capability working end-to-end first (Planner), then add the others one at a time; put the vague-input rule in every prompt from the start; connect tracing early, not the night before. Fine-tuning is reserved for Phase 4/5 and only if evaluation proves prompting has plateaued.